

# Automated scale up and down of an Azure App Service plan

AZURE AUTOMATION RUNBOOKS, A MANAGED IDENTITY AND AZURE MONITOR ALERTS

---

## Summary

This document describes vertical scaling of an Azure App Service plan, automated with Azure Automation and a managed identity. Vertical scaling changes the plan's SKU tier up or down in response to performance metrics such as CPU or memory usage. Azure Monitor alerts trigger PowerShell runbooks in an Automation account, and the runbooks use the account's managed identity to change the plan's SKU.

The aim is to remove manual intervention, keep cost and performance in balance, and keep the application available under variable load. A short troubleshooting guide is at the end.

---

## How it fits together

- An Automation account with a system-assigned managed identity.
- The managed identity holds the Contributor role on the App Service plan or plans.
- Two PowerShell runbooks: ScaleUp and ScaleDown.
- ScaleUp moves the SKU one level up in a predefined list.
- ScaleDown moves it one level down, and retries if the operation is queued.
- Azure Monitor metric alerts (for example CPU above 90% triggers ScaleUp; CPU below 40% triggers ScaleDown).
- The alerts invoke the runbooks through action groups.

This gives basic vertical scaling automation. Its limits are in flexibility, security and observability, which is where it can be improved next.

---

## Create the Automation account

1. Create an Automation account to hold the runbooks that scale up and down. Its managed identity must be **system assigned**, and the Az modules must be installed in the account.
  2. Assign the account's managed identity the Contributor role on every App Service plan that needs the feature: copy the identity's object ID and add it under each plan's access control.
  3. In the Automation account, go to Process automation > Runbooks and create a runbook.
  4. Enter a name, for example `ScaleUpWebApp`. Select PowerShell as the runbook type. The runtime version is 5.1, which Azure Automation supports and which is the most stable.
  5. Review and create the runbook. An editor opens in the portal; paste the script.
- 

## Scale-up runbook

Each trigger moves up one tier only, which keeps scaling smooth.

```

<#
.SYNOPSIS
Scale UP an App Service plan (P0v4 > P1v4 > P2v4 > P3v4)
#>

# Config
$ResourceGroupName = "<resource-group>"
$PlanName           = "<app-service-plan>"
$SubscriptionId     = "<subscription-id>"

# Upgrade chain
$SkuUpgradeOrder = @("P0v4", "P1v4", "P1mv4", "P2v4", "P2mv4", "P3v4", "P3mv4", "P4mv4", "P5mv4")

Write-Output "=== Starting Scale-UP for: $PlanName ==="

try {
    Connect-AzAccount -Identity -ErrorAction Stop | Out-Null
    Write-Output "Authenticated with Managed Identity."

    Set-AzContext -SubscriptionId $SubscriptionId -ErrorAction Stop | Out-Null
    Write-Output "Context set to Subscription: $SubscriptionId"

    $plan = Get-AzAppServicePlan -ResourceGroupName $ResourceGroupName -Name $PlanName -ErrorAction Stop
    $currentSku = $plan.Sku.Name
    Write-Output "Current SKU: $currentSku"

    $index = $SkuUpgradeOrder.IndexOf($currentSku)
    if ($index -eq -1) {
        Write-Output "Unknown SKU: $currentSku. Exiting."
        exit 0
    }

    if ($index -lt ($SkuUpgradeOrder.Count - 1)) {
        $newSku = $SkuUpgradeOrder[$index + 1]
        Write-Output "Scaling UP from $currentSku to $newSku ..."

        # Modify the SKU on the object: PowerShell 5.1 lacks the newer parameters
        $plan.Sku.Name = $newSku
        $plan.Sku.Tier = "PremiumV4"
        $plan | Set-AzAppServicePlan -ErrorAction Stop

        Write-Output "Successfully scaled UP to $newSku"
    }
    else {
        Write-Output "Already at maximum tier ($currentSku). Nothing to do."
    }
}
catch {
    Write-Output "Scale operation failed: $_"
}

Write-Output "=== Scale-UP Script Completed ==="

```

## Scale-down runbook

Each trigger moves down one tier only. A scale-down can also fail even though the runbook reports success: the request is queued on Microsoft's side, and if no capacity is free in the target SKU the change does not happen. So the scale-down retries over a period, and checks the plan afterwards, to make sure the change really took effect.

```

<#
.SYNOPSIS
Scale DOWN an App Service plan (P3v4 > P2v4 > P1v4 > P0v4)
#>

# Config
$ResourceGroupName = "<resource-group>"
$PlanName           = "<app-service-plan>"
$SubscriptionId     = "<subscription-id>"
$MaxAttempts        = 5
$RetryDelaySeconds  = 60

# Downgrade chain (reverse of scale-up)
$SkuDowngradeOrder = @("P3v4", "P2v4", "P1v4", "P0v4")

Write-Output "=== Starting Scale-DOWN for: $PlanName ==="

try {
    Connect-AzAccount -Identity -ErrorAction Stop | Out-Null
    Write-Output "Authenticated with Managed Identity."

    Set-AzContext -SubscriptionId $SubscriptionId -ErrorAction Stop | Out-Null
    Write-Output "Context set to Subscription: $SubscriptionId"

    $plan = Get-AzAppServicePlan -ResourceGroupName $ResourceGroupName -Name $PlanName -ErrorAction Stop
    $currentSku = $plan.Sku.Name
    Write-Output "Current SKU: $currentSku"

    $index = $SkuDowngradeOrder.IndexOf($currentSku)
    if ($index -eq -1) {
        Write-Output "Unknown SKU: $currentSku. Exiting."
        exit 0
    }

    if ($index -lt ($SkuDowngradeOrder.Count - 1)) {
        $newSku = $SkuDowngradeOrder[$index + 1]

        for ($attempt = 1; $attempt -le $MaxAttempts; $attempt++) {
            Write-Output ("Attempt {0}: Scaling DOWN from {1} to {2} ..." -f $attempt, $currentSku,
$newSku)
            try {
                $plan.Sku.Name = $newSku
                $plan.Sku.Tier = "PremiumV4"
                $plan | Set-AzAppServicePlan -ErrorAction Stop | Out-Null
            }
            catch {
                Write-Output "Attempt $attempt failed: $_"
            }

            # The request can be queued: confirm the plan really changed.
            $plan = Get-AzAppServicePlan -ResourceGroupName $ResourceGroupName -Name $PlanName -ErrorAction
Stop
            if ($plan.Sku.Name -eq $newSku) {
                Write-Output "Successfully scaled DOWN to $newSku"
                break
            }
            if ($attempt -lt $MaxAttempts) { Start-Sleep -Seconds $RetryDelaySeconds }
        }

        if ($plan.Sku.Name -ne $newSku) {
            Write-Output "Scale-DOWN did not take effect after $MaxAttempts attempts. Plan is still
$(($plan.Sku.Name))."
        }
    }
}

```

```
else {
    Write-Output "Already at minimum tier ($currentSku). Nothing to do."
}
}
catch {
    Write-Output "Scale operation failed: $_"
}

Write-Output "=== Scale-DOWN Script Completed ==="
```

---

## Create an action group

After creating both runbooks for an App Service plan, create the action groups that will call them.

1. Navigate to the App Service plan > Monitoring > Alerts > Action groups.
2. Create an action group. On the Actions tab, select the action type **Automation Runbook**. In the pane on the right, select the runbook source **User**, then choose the subscription, the Automation account created earlier, and the runbook.
3. The scripts take no parameters, so leave those unchanged. Click Done, then review and create the action group.

Action groups are shared globally. With two different applications to scale up, you need two scale-up action groups, and the same again for scale-down: four in that case.

Before wiring an alert to it, test the action group. Open it and click **Test**, next to Edit and Disable. In the pane that opens, choose the metric-based static sample from the dropdown, because the trigger will be a metric. Check the result in the runbook's Jobs list, or in the plan's pricing tier on its Overview page.

---

## Create the alerts

1. Navigate to Alerts on the App Service plan and create a new alert rule.
2. On the Conditions tab, set the condition: for example, CPU percentage greater than 90% over a lookback period of 5 minutes, checked every 1 minute.
3. On the Actions tab, select the scale-up action group.
4. Repeat for scale-down with its own threshold, for example below 40%, and the scale-down action group.

Set the lookback period and the check frequency generously enough that the actions are not triggered again and again.

---

## Dashboard

A dashboard in the Azure portal, shared from the production resource group, shows the server's CPU, memory, network traffic, HTTP requests and latency in one place. Add further metrics as needed through the Edit option under the dashboard's name.

---

## Troubleshooting

**"Unauthorized" during a change to or from an m-series SKU.** The region may have no quota assigned for the memory-optimized (m) series. Either raise a support request with Microsoft to have m-series quota assigned in the region, or remove the m-series SKUs from the upgrade list and run only the general v4 series. The "m" stands for memory: those SKUs give the plan more of it.